

STEP 2 · 3주차

모델 비교와 통계 검정

세 모델을 같은 조건에서 겨루게 하고 차이를 검정한다

강의 50분

용어 정리

실습 6단계

이번 주의 핵심 문장 — 수치가 높다고 더 좋은 모델이라고 말할 수는 없다.

이번 주 진행 방식

마지막 주

① 강의

50분

SVM · RF · XGBoost · 검증 · 검정

본 강의 자료 31~52페이지

② 용어 정리

20분

모델과 검증 용어

이 자료 4~7페이지

③ 실습

70분

학습 → 평가 → 교차검증 → 검정

이 자료 9~15페이지

④ 최종 제출

3주차 자정

리포트와 코드 저장소

이 자료 18페이지

지난주 `dataset.npz` 를 만들어 두었다면 오늘 실습은 그 파일을 불러오는 것부터 시작한다.

오늘 강의에서 기억할 여섯 가지

실습 전에 다시 확인

- | | | |
|---|----------------------|---------------------------------------|
| 1 | SVM | RBF 커널로 비선형 경계를 그린다. 표준화가 반드시 필요하다 |
| 2 | Random Forest | 결정 트리 100개의 다수결. 분산을 줄여 과적합에 강하다 |
| 3 | XGBoost | 앞선 트리의 오차를 다음 트리가 보정한다. 학습이 느리다 |
| 4 | 두 가지 검증 | hold-out으로 빠르게, 5-fold로 안정성까지 본다 |
| 5 | 통계 검정 | 세 모델의 차이가 우연인지 대응표본 t-검정으로 판단한다 |
| 6 | 논문의 결론 | RF가 96.72%로 1위였지만 세 모델 간 차이는 유의하지 않았다 |

마지막 줄이 이 논문에서 가장 중요한 교훈이다. 1위라는 사실만으로는 주장이 되지 않는다.

용어 정리 ① 세 가지 모델

각각 무엇이 다른가

서포트 벡터 머신

SVM

클래스를 가르는 최대 마진 경계를 찾는다

→ RBF 커널 · 표준화 필수

커널

Kernel

데이터를 고차원으로 보내 비선형 경계를 만드는 기법

→ rbf = 가장 널리 쓰이는 기본값

랜덤 포레스트

Random Forest

여러 결정 트리의 다수결 (앙상블)

→ 트리 100개 · 분산 감소

부스팅

Boosting

앞 모델의 오차를 다음 모델이 보정

→ XGBoost의 학습 방식

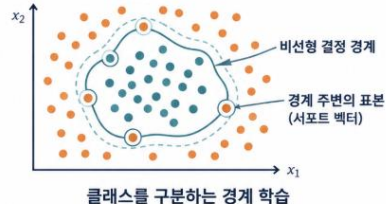
앙상블

Ensemble

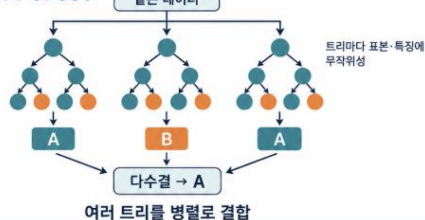
여러 모델을 묶어 하나처럼 쓰는 것

→ RF와 XGBoost 모두 앙상블

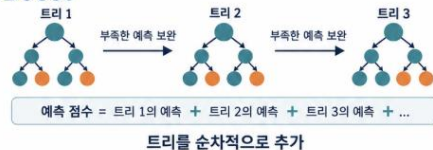
SVM · RBF 커널



Random Forest



XGBoost



세 모델은 데이터를 나누는 방식이 근본적으로 다르다. 그래서 같은 데이터에서도 결과가 달라진다.

용어 정리 ② 하이퍼파라미터

모델마다 정해줘야 하는 값

하이퍼파라미터

Hyperparameter

학습 전에 사람이 정하는 값

→ 트리 개수 · 커널 종류 등

n_estimators

-

앙상블에 쓸 트리의 개수

→ RF는 100, XGBoost는 400

max_depth

-

트리를 얼마나 깊게 키울지

→ XGBoost는 6

learning_rate

-

한 번에 얼마나 크게 고칠지

→ XGBoost는 0.05 (보수적)

random_state

-

무작위성을 고정하는 씨앗값

→ 42 — 재현성 확보

하이퍼파라미터



학습 전에 정하는 설정

개수 · 깊이 · 학습률

n_estimators



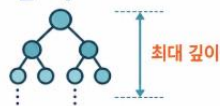
RF: 100

XGBoost: 400

트리 수를 정하는 설정

XGBoost: 부스팅 반복 수

max_depth



XGBoost: 6

분기 깊이 제한

learning_rate



XGBoost: 0.05

추가 예측 × 0.05

새 트리의 기여도를 조절

random_state



같은 조건에서 재현성

환경 · 데이터 · 설정도 같아야 함

논문은 SVM·RF는 기본값을, XGBoost만 예비 튜닝을 했다. 완전히 대칭적인 비교는 아니라는 점을 기억할 것.

용어 정리 ③ 평가와 검증

숫자를 믿을 수 있게 만들기

홀드아웃

Hold-out

한 번 나눠 학습·평가

→ 빠르지만 분할 운에 좌우

교차검증

Cross-validation

여러 번 나눠 평균낸다

→ 5-fold 사용

층화 K-fold

StratifiedKFold

각 폴드에 클래스 비율 유지

→ 8클래스가 고르게

Macro F1

-

클래스별 F1의 단순 평균

→ 소수 클래스를 동등하게 취급

표준편차

Standard deviation

폴드 간 성능의 흔들림

→ 이번 폴드 간 변동을 나타냄

향후 검증: LOSO

1

2

3

4

5

6

7

8

7명 학습 / 남은 1명 평가
평가 피험자를 바꾸어 반복

논문은 trial-wise 결과만 보고
LOSO 성능은 후속 연구 과제

평균만 쓰면 절반만 보고한 것이다. 표준편차를 반드시 함께 적을 것.

용어 정리 ④ 통계 검정

귀무가설 아래 차이의 증거 평가

가설 검정

Hypothesis test

자료가 귀무가설과 얼마나 부합하는지 평가

→ 모델 비교에 사용

대응표본 t-검정

Paired t-test

같은 조건에서 짝지어 비교

→ 폴드별 정확도를 짝으로

p-value

-

귀무가설 아래 관찰 이상으로 극단적인 통계량의 확률

→ 0.05 미만이면 유의하다고 본다

유의수준

Significance level (α)

판단 기준선

→ 보통 0.05

비모수 검정

Non-parametric

정규분포를 가정하지 않는 검정

→ 작은 표본에서 항상 더 안전한 것은 아님

1 가설 검정



H_0 : 평균 차이 = 0



자료가 H_0 와 얼마나 부합하는가

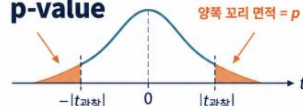
2 대응표본 t-검정

	모델 A	모델 B	
Fold 1	●	●	→ 차이 d
Fold 2	●	●	→ 차이 d
Fold 3	●	●	→ 차이 d
⋮	⋮	⋮	⋮

같은 폴드끼리 짝지어 비교

검정 가정·폴드 의존성 확인

3 p-value



H_0 아래 관찰 이상으로 극단적인 결과

양측 검정 예시

4 유의수준 $\alpha = 0.05$

$p < 0.05$ → 유의한 차이
 $p \geq 0.05$ → 유의한 차이 미검출



미검출 ≠ 등등성 입증

5 비모수 검정



검정에 따라 순위·부호 활용

정규성 가정이 덜 필요할 수 있음
 작은 표본에서 항상 더 안전한 것은 아님

$p < 0.05$ 면 유의, 그보다 크면 차이를 주장할 수 없다. 이 논문은 모두 0.05를 넘었다.

오늘의 실습

여섯 단계 — 지난주 데이터셋에서 시작한다

단계 1

세 모델 학습

같은 데이터·같은 분할로

단계 2

hold-out 평가

네 지표를 한 번에

단계 3

혼동행렬

8클래스 오류 지도

단계 4

5-fold 교차검증

이번 5개 분할에서 변동을 확인한다

단계 5

통계 검정

차이가 우연인지 판단

단계 6

계산 비용과 리포트

정확도 외의 측을 측정

교차검증은 학습을 5번 반복하므로 시간이 걸린다. 먼저 돌려놓고 다른 단계를 진행할 것.

세 모델 학습

조건이 하나라도 다르면 비교가 성립하지 않는다

목표 SVM · RF · XGBoost를 동일 조건에서 학습 한다

```
import numpy as np
from sklearn.svm import SVC
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
d = np.load('dataset.npz', allow_pickle=True)
Xtr, ytr, Xte, yte = d['Xtr'], d['ytr'], d['Xte'], d['yte']
le = LabelEncoder(); ytr = le.fit_transform(ytr)
yte = le.transform(yte)
models = {
    'SVM': SVC(kernel='rbf', C=1.0, gamma='scale',
               decision_function_shape='ovr'),
    'RF' : RandomForestClassifier(n_estimators=100,
                                random_state=42, n_jobs=-1),
    'XGB': XGBClassifier(n_estimators=400, max_depth=6,
                        learning_rate=0.05, subsample=0.9,
                        colsample_bytree=0.8, random_state=42),
}
for k, m in models.items():
    m.fit(Xtr, ytr); print(k, '학습 완료')
```

확인 방법

- 세 줄의 학습 완료가 출력되면 성공
- XGBoost는 라벨을 0부터 시작하는 정수로 바꿔야 한다
- SVM이 오래 걸리면 샘플 수를 먼저 확인
- random_state 를 고정해야 다음에도 같은 결과

hold-out 평가

정확도만 보지 않는다

목표 정확도 · 정밀도 · 재현율 · F1을 세 모델에 대해 산출한다

```
from sklearn.metrics import (accuracy_score,
                             precision_score, recall_score, f1_score)

rows = []
for k, m in models.items():
    p = m.predict(Xte)
    rows.append([k,
                 accuracy_score(yte, p),
                 precision_score(yte, p, average='macro'),
                 recall_score(yte, p, average='macro'),
                 f1_score(yte, p, average='macro')])

import pandas as pd
df = pd.DataFrame(rows, columns=
    ['model', 'acc', 'precision', 'recall', 'f1'])
print(df.round(4))
```

확인 방법

- 세 행 네 열의 표가 출력되면 성공
- 지표가 비슷해도 클래스별 오류는 따로 확인
- 논문 Table 2와 비교해 볼 것
- macro 평균은 실습 선택임을 리포트에 명시

혼동행렬

동작 간 오류와 각도 간 오류를 구분한다

목표 어떤 클래스끼리 헷갈리는지 찾고 그 구조를 해석한다

```
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

L = le.classes_ # 실제 인코딩 순서
cm = confusion_matrix(yte, models['RF'].predict(Xte))

fig, ax = plt.subplots(figsize=(6, 5))
im = ax.imshow(cm, cmap='Blues')
ax.set_xticks(range(8), L, rotation=45)
ax.set_yticks(range(8), L)
for i in range(8):
    for j in range(8):
        if cm[i,j]: ax.text(j, i, cm[i,j], ha='center',
                             va='center', fontsize=8)
plt.colorbar(im); plt.tight_layout()
plt.savefig('confusion8.png', dpi=120)
```

확인 방법

- 대각선이 진하게 나오면 정상
- 행.열의 실제 동작과 각도 라벨을 대조한다
- 인접 칸이라는 이유만으로 오류 종류를 단정하지 않는다
- 논문은 같은 동작의 두 각도 간 혼동을 주요 오류로 설명

5-fold 교차검증

평균과 표준편차를 함께 구한다

목표 세 모델의 5-fold 평균 정확도와 표준편차를 산출한다

```
from sklearn.model_selection import StratifiedKFold
from sklearn.base import clone
from sklearn.preprocessing import StandardScaler
skf = StratifiedKFold(5, shuffle=True, random_state=42)
fold_acc = {k: [] for k in models}
# 주의: 시행(파일) 단위로 나눈다
for tr_i, va_i in skf.split(files, labels):
    Xa, ya = build([files[i] for i in tr_i])
    Xb, yb = build([files[i] for i in va_i])
    sc = StandardScaler(); Xa = sc.fit_transform(Xa)
    Xb = sc.transform(Xb); ya, yb = le.transform(ya), le.transform(yb)
    for k, proto in models.items():
        m = clone(proto); m.fit(Xa, ya)
        fold_acc[k].append(m.score(Xb, yb))
for k, v in fold_acc.items():
    print(f'{k}: {np.mean(v)*100:.2f} ± {np.std(v)*100:.2f} %')
```

확인 방법

- 세 줄이 평균 $\pm$ 표준편차 형태로 출력되면 성공
- sklearn.base 의 clone 을 import 해야 한다
- 각 학습 폴드에서만 scaler를 fit한다
- 논문 결과는 RF 96.72 ± 0.58 이었다

통계 검정

이번 주 실습의 하이라이트

목표 폴드별 정확도로 대응표본 t-검정을 수행한다

```
from scipy.stats import ttest_rel
from itertools import combinations

for a, b in combinations(models.keys(), 2):
    t, p = ttest_rel(fold_acc[a], fold_acc[b])
    if not np.isfinite(p): print(a, b, '검정 불가'); continue
    verdict = '유의함' if p < 0.05 else '유의하지 않음'
    print(f'{a} vs {b}: p = {p:.4f} ({verdict})')
```

확인 방법

- 세 쌍의 p값이 출력되면 성공
- $p \geq 0.05$ 는 차이 미검출이며 동등성의 증거는 아님
- 논문에서는 세 쌍 모두 유의하지 않았다
- 내 결과가 다르면 그 이유를 리포트에 적을 것

계산 비용과 리포트

무엇으로 모델을 고를 것인가

목표 학습 시간과 추론 시간을 재고 리포트 구조에 맞춰 정리한다

```
import time

for k, m in models.items():
    t0 = time.time(); m.fit(Xtr, ytr)
    train_t = time.time() - t0

    t0 = time.time()
    for _ in range(100): m.predict(Xte[:1])
    infer_ms = (time.time() - t0) * 10

    print(f'{k}: 학습 {train_t:.3f}s, '
          f'추론 {infer_ms:.3f} ms/sample')
```

확인 방법

- 논문 측정에서는 XGBoost 학습이 가장 느렸다
- 논문 측정에서는 SVM 추론이 상대적으로 느렸다
- 세 모델의 정확도가 비슷하다면 이 값이 선택 기준이 된다
- 측정 환경 차이를 기록하며 실제 배포 성능과 구분

여기까지 됐으면 정상

스스로 확인하는 체크포인트

오늘 안에 확인할 것

- 세 모델을 같은 데이터·같은 분할로 학습했다
- hold-out 네 지표 표를 만들었다
- confusion8.png 를 만들고 오류 구조를 해석했다
- 5-fold 평균 $\pm$ 표준편차를 산출했다
- 대응표본 t-검정으로 p값을 구했다
- 학습·추론 시간을 측정했다
- 한계 항목에 trial-wise 분할 문제를 적었다

마지막 항목을 빠뜨리지 말 것. 이 결과는 처음 보는 사람에 대한 성능이 아니다.

자주 나는 오류와 해결

평가 단계의 함정들

증상	원인	해결
XGBoost 라벨 오류	문자열 라벨을 그대로 넣음	LabelEncoder 로 0부터 정수 변환
정확도가 99% 이상	같은 시행의 겹친 창·라벨 유입 등 확인 필요	2주차 단계 5의 순서 확인
clone 이 정의되지 않음	import 누락	from sklearn.base import clone
교차검증이 너무 느림	매 폴드마다 특징을 새로 계산	특징을 미리 만들고 인덱스로 나누기
p값이 nan	폴드 간 차이의 분산이 0 등 확인	차이 벡터·유한값·검정 한계를 확인
모델 순위가 매번 바뀜	random_state 미고정	모든 모델에 random_state=42

높은 정확도나 NaN만으로 누수를 단정하지 않는다. 분할과 검정 조건을 확인한다.

최종 제출물

Step 2 마무리

① 리포트	7개 항목 구조를 따른 PDF 1개
② 성능 비교표	세 모델 × 네 지표 (hold-out)
③ 교차검증 결과	폴드별 값 + 평균 ± 표준편차
④ 통계 검정	세 쌍의 p값과 해석 문장
⑤ 혼동행렬	confusion8.png + 오류 구조 해석
⑥ 계산 비용	학습 시간 · 추론 시간 표
⑦ 한계 서술	trial-wise 분할 문제를 포함해 세 가지 이상
제출 기한	3주차 강의일 자정 · 지각 시 24시간마다 10% 감점

④번의 해석 문장이 중요하다. p값 숫자만 적고 끝내면 절반만 한 것이다.

Step 2를 마치며

무엇을 얻었는가

3주 동안 익힌 것

- 다차원 의도를 클래스 집합으로 설계
- 시간영역 특징 5종 직접 구현
- SVM · RF · XGBoost의 원리와 설정
- 교차검증과 통계 검정 절차
- 분할 단위가 결과 해석을 좌우한다

Step 3으로 이어지는 것

- 데이터 누수 방지 원칙
- 베이스라인 비교 습관
- 혼동행렬 기반 오류 분석
- 계산 비용까지 함께 보고

Step 3 예고 — 지하구조물 결함 탐지

- 생체신호를 떠나 이미지로 넘어간다. 시계열에서 익힌 원칙이 컴퓨터 비전에서 어떻게 다시 나타나는지 확인한다
- 다음 주 1주차 실습에서는 ultralytics 패키지를 설치한다
- GPU 서버 계정이 아직 없다면 이번 주에 반드시 신청할 것



DAEGU
CATHOLIC
UNIVERSITY



Thank you



[Http://dil.cu.ac.kr/](http://dil.cu.ac.kr/) 데이터 지능 연구실